Presentación Proyecto: Taller mecánico

1- Integrantes:

2- Descripción breve del proyecto

Sección 2 - Descripción breve

Describir con un máx. de 200 a 300 palabras la solución a desarrollar, a quién está destinada, principal aporte y resultado concreto.

Vamos a desarrollar el backend de un sistema de gestión interna para un taller mecánico de autos. La idea es cubrir todo el recorrido de un vehículo, desde la llegada del vehículo hasta que fue retirado por el propietario, pasando por el diagnóstico, el presupuesto, el pedido de repuestos y la reparación. El sistema está pensado para el personal del taller. El jefe de taller es quien registra al cliente y al vehículo a partir de la patente. También arma el presupuesto con el costo de repuestos y mano de obra, registra si el cliente lo acepta o lo rechaza, pide los repuestos y avisa por WhatsApp que el auto está para retirar. El mecánico asignado registra el VIN del vehículo, que se consulta en la API vPIC para traer marca, modelo, año y motor (si no lo encuentra, se carga a mano). Además anota los desperfectos que detecta, con el escáner o de forma manual, y los repuestos que hacen falta. Una vez aprobado el arreglo, puede agregar nuevas fallas o repuestos que aparezcan durante la reparación, y eso genera un presupuesto adicional. El cliente solo interviene para aceptar o rechazar el presupuesto. El principal aporte es que el sistema no solo guarda datos, sino que controla el proceso de la OT con reglas. No se repara ni se piden repuestos sin que el cliente acepte, y el presupuesto vence a los 7 días. Además, solo el mecánico asignado puede cargar información en la orden. Como resultado, el taller va a poder seguir cada orden de trabajo de punta a punta, saber en qué estado está cada auto y mantener al cliente informado sin tener que llamarlo

3- Problema que se busca resolver

Sección 3 - Problema y contexto

Describan el problema actual, sus causas o dificultades principales, a quiénes afecta, qué actores intervienen y por qué resulta pertinente resolverlo mediante el sistema propuesto. Diferencien el problema de la solución técnica.

En muchos talleres mecánicos el seguimiento de cada vehículo se hace de manera informal, con planillas en papel, anotaciones sueltas y mensajes desde el celular personal. El problema es que el taller no tiene un registro ordenado de en qué estado está cada auto, qué se le detectó, qué se presupuestó, qué aprobó el cliente y qué falta hacer.

Las principales causas son que la información se reparte entre el jefe de taller y los mecánicos y muchas veces se pasa de palabra. Además, los presupuestos se arman a mano y sin vencimiento claro, y las fallas que aparecen durante la reparación no siempre quedan registradas como autorizadas. A esto se suma que los datos del vehículo se cargan a mano, con riesgo de error, y que avisarle al cliente depende de que alguien se acuerde.

Esto afecta principalmente al taller. El jefe de taller pierde tiempo buscando información, los mecánicos pueden trabajar sobre algo no aprobado o quedar frenados sin saberlo, y aparecen reclamos del cliente por montos o trabajos que no recuerda haber aceptado.

Intervienen el jefe de taller, que organiza el trabajo, presupuesta y entrega. Los mecánicos, que diagnostican y reparan. El cliente que no opera el sistema pero recibe los avisos y decide sobre el presupuesto.

Resolverlo con el sistema propuesto es pertinente porque el problema no es de falta de conocimiento técnico, sino de organización y trazabilidad del proceso. El problema es la falta de control sobre el recorrido de cada orden de trabajo. La solución técnica, un backend que registra cada etapa, aplica reglas antes de dejar avanzar la orden, obtiene los datos del vehículo desde una fuente externa y avisa al cliente en los momentos clave, es solo la herramienta para atacar ese problema.

4- Alcance funcional y proceso principal

Sección 4 - Alcance funcional y proceso principal

1) Funcionalidades: identificador, nombre, actor, comportamiento, resultado y prioridad.
2) Reglas de negocio: identificador, condición, comportamiento, rechazos y funcionalidad asociada.
3) Proceso principal: Desde evento inicial a final, participantes, pasos, decisiones, excepciones y resultado final. Más BPMN y estados.

Alcance del MVP: El sistema modela solo el trabajo interno del taller sobre una orden de trabajo (OT), desde el ingreso del vehículo hasta su entrega. Lo usan el jefe de taller y el mecánico. El cliente no opera el sistema, recibe por WhatsApp el presupuesto y el aviso de retiro, y su decisión la registra el jefe de taller.

Funcionalidades

F01) Registrar cliente y vehículo. Actor: jefe de taller. Comportamiento: busca por patente y, si no existe, los da de alta. Resultado: cliente y vehículo registrados. Prioridad: alta.

F02) Abrir OT y asignar mecánico. Actor: jefe de taller. Comportamiento: crea la OT y asigna un mecánico. Resultado: OT Recibida. Prioridad: alta.

F03) Registrar VIN. Actor: mecánico asignado. Comportamiento: consulta vPIC y, si falla, permite carga manual. Resultado: datos del vehículo guardados y OT En diagnóstico. Prioridad: alta.

F04) Registrar desperfectos. Actor: mecánico asignado. Comportamiento: carga códigos del escáner o una descripción manual. Resultado: desperfectos en la OT. Prioridad: alta.

F05) Registrar repuestos necesarios. Actor: mecánico asignado. Comportamiento: indica repuesto y cantidad. Resultado: lista de repuestos de la OT. Prioridad: alta.

F06) Generar presupuesto. Actor: jefe de taller. Comportamiento: calcula repuestos, mano de obra y otros cargos, con vencimiento a 7 días. Resultado: OT Presupuestada y WhatsApp al cliente. Prioridad: alta.

F07) Registrar decisión del cliente. Actor: jefe de taller. Comportamiento: registra si el cliente aceptó o rechazó. Resultado: OT Aceptada o Rechazada, visible para el mecánico. Prioridad: alta.

F08) Gestionar repuestos. Actor: jefe de taller. Comportamiento: registra el pedido y la llegada de los repuestos. Resultado: OT Esperando repuestos y luego En reparación. Prioridad: alta.

F09) Agregar anotación durante la reparación. Actor: mecánico asignado. Comportamiento: agrega una falla o un repuesto nuevo. Resultado: OT en Aprobación adicional. Prioridad: media.

F10) Finalizar trabajo. Actor: mecánico asignado. Comportamiento: marca el trabajo terminado. Resultado: OT Lista para retirar y WhatsApp al cliente. Prioridad: alta.

F11) Registrar entrega. Actor: jefe de taller. Comportamiento: registra el retiro del vehículo. Resultado: OT Entregada. Prioridad: alta.

F12) Enviar notificaciones. Actor: sistema. Comportamiento: envía por WhatsApp el presupuesto y el aviso de retiro, y reintenta si falla. Resultado: historial de mensajes. Prioridad: media.

Fuera del alcance: las acciones del cliente, los turnos, la facturación, el stock y las compras a proveedores, varias sucursales y el frontend.

Reglas de negocio

RN-01) Condición: se registra un vehículo. Comportamiento: la patente debe tener formato AAA123 o AA123AA y no repetirse. Rechazo: patente inválida o duplicada. Funcionalidad: F01.

RN-02) Condición: se abre una OT. Comportamiento: un vehículo no puede tener dos OT abiertas. Rechazo: OT ya en curso. Funcionalidad: F02.

RN-03) Condición: un mecánico carga datos en una OT. Comportamiento: solo el mecánico asignado puede hacerlo. Rechazo: mecánico no asignado. Funcionalidad: F03, F04, F05, F09 y F10.

RN-04) Condición: se registra el VIN. Comportamiento: debe tener 17 caracteres sin I, O ni Q; si la marca de vPIC no coincide con la registrada, el presupuesto se bloquea. Rechazo: VIN inválido o inconsistente. Funcionalidad: F03 y F06.

RN-05) Condición: se registra un desperfecto. Comportamiento: si viene del escáner debe ser un código OBD-II válido; si es manual, requiere descripción. Rechazo: código o descripción inválidos. Funcionalidad: F04.

RN-06) Condición: se genera o se acepta un presupuesto. Comportamiento: total = repuestos + mano de obra + otros cargos; requiere al menos un desperfecto y vence a los 7 días. Rechazo: sin desperfectos o vencido. Funcionalidad: F06 y F07.

RN-07) Condición: se piden repuestos o se empieza a reparar. Comportamiento: requiere la aceptación registrada y que hayan llegado todos los repuestos aceptados. Rechazo: sin aceptación o con repuestos pendientes. Funcionalidad: F07 y F08.

RN-08) Condición: aparece una falla o un repuesto nuevo. Comportamiento: se genera un presupuesto adicional y lo nuevo no se ejecuta hasta que se acepta. Rechazo: ejecutar lo nuevo sin aceptación. Funcionalidad: F09, F06 y F07.

RN-09) Condición: se genera un presupuesto o la OT queda Lista para retirar. Comportamiento: se envía un WhatsApp; si falla, se reintenta sin frenar la OT. Rechazo: ninguno. Funcionalidad: F12.

RN-10) Condición: se finaliza o se entrega. Comportamiento: todo lo aceptado debe estar terminado y no puede haber un adicional pendiente. Rechazo: tareas o adicional pendientes. Funcionalidad: F10 y F11. Estados de la OT: Recibida, En diagnóstico, Presupuestada, Aceptada, Rechazada, Esperando repuestos, En reparación, Aprobación adicional, Lista para retirar y Entregada. Las transiciones se ven en el diagrama.

Proceso principal

Evento inicial: el vehículo ingresa al taller. Participantes: el jefe de taller y el mecánico, y como sistemas externos vPIC y WhatsApp.

Paso 1: el jefe de taller registra al cliente y al vehículo, abre la OT y asigna un mecánico (F01, F02, RN-01, RN-02).

Paso 2: el mecánico registra el VIN, los desperfectos y los repuestos (F03, F04, F05, RN-03, RN-04, RN-05).

Paso 3: el jefe de taller genera el presupuesto y se envía por WhatsApp (F06, F12, RN-06, RN-09).

Paso 4: el jefe de taller registra la decisión del cliente (F07). Si la rechazó, el vehículo se devuelve sin reparar.

Paso 5: si hacen falta repuestos, el jefe de taller los pide y registra su llegada (F08, RN-07).

Paso 6: el mecánico repara. Si aparece algo nuevo, lo agrega y se repiten los pasos 3 y 4 para esa parte (F09, RN-08).

Paso 7: el mecánico finaliza el trabajo y se avisa al cliente por WhatsApp (F10, F12, RN-10).

Paso 8: el jefe de taller registra la entrega (F11, RN-10). Excepciones: si vPIC falla, los datos se cargan a mano como no verificados; si el presupuesto vence, se genera uno nuevo; si WhatsApp falla, el aviso se reintenta. Resultado final: el vehículo entregado y la OT cerrada con su historial completo. Diagrama de estados (Mermaid):

El diagrama modela el ciclo de vida de la orden de trabajo, la entidad que recorre todo el proceso principal. Cada transición usa la notación UML *evento [guarda] / acción* e indica la funcionalidad (F) y las reglas de negocio (RN) que la controlan. Así se ve qué operación mueve la OT de un estado a otro y en qué condiciones se rechaza.

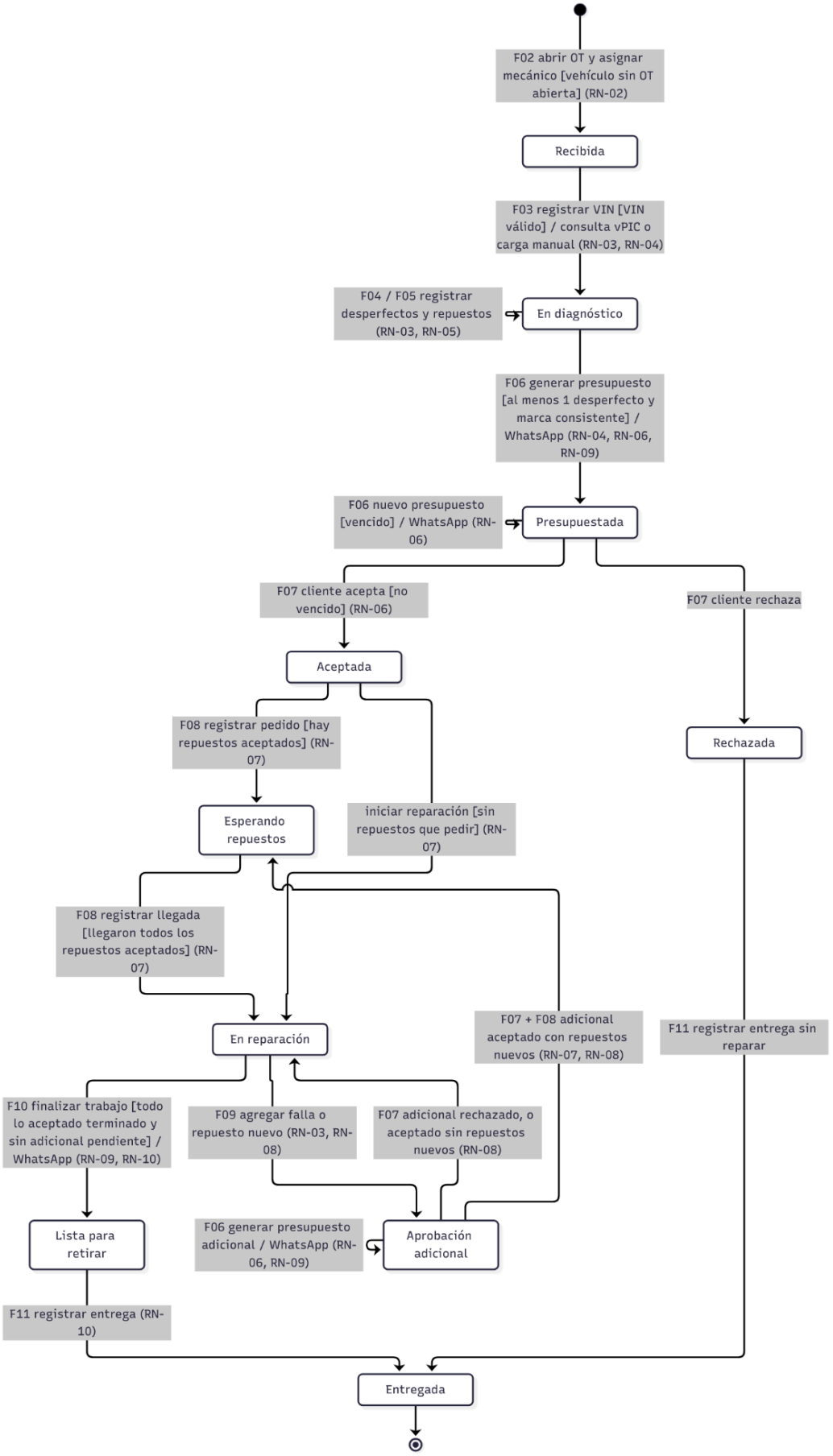
- **Camino principal:** Recibida → En diagnóstico → Presupuestada → Aceptada → Esperando repuestos (si hacen falta) → En reparación → Lista para retirar → Entregada.
- **Rechazo:** si el cliente no acepta, la OT pasa a Rechazada. Se cierra cuando se registra la entrega del vehículo sin reparar.
- **Vencimiento:** un presupuesto vencido no se puede aceptar. Se genera uno nuevo y la OT sigue en Presupuestada (RN-06).
- **Trabajo adicional:** si en la reparación aparece una falla o un repuesto nuevo, la OT pasa a Aprobación adicional. Se presupuesta y se notifica de nuevo, y lo nuevo solo se hace si se acepta (RN-08). Mientras esté pendiente, la OT no se puede finalizar (RN-10).
- **Notificaciones:** WhatsApp es una acción de las transiciones a Presupuestada, Lista para retirar y del presupuesto adicional. Si falla, se reintenta sin frenar la transición (RN-09).
- **Auto Transiciones:** cargar desperfectos y repuestos (F04/F05) agrega datos a la OT pero no le cambia el estado.

En el backend, cada operación que dispara una transición controla el estado de origen y la guarda. Si alguno no se cumple, rechaza la operación y la OT queda como estaba.

Dos decisiones que tomé y conviene que confirmen con el equipo:

- **Aceptada → En reparación:** cuando no hay repuestos que pedir, ninguna funcionalidad del compromiso cubre esa transición. La etiqueté “iniciar reparación”. Hay que definir quién la dispara, o hacerla automática al aceptar.
- **Rechazada → Entregada:** hice que la OT rechazada se cierre con F11 para que el vehículo no quede con una OT abierta (RN-02). Si prefieren que Rechazada sea un estado final, se cambia una línea.

Ciclo de vida de la Orden de Trabajo (OT)



5- Diagrama C4 - Nivel 1 Contexto

Sección 5 - Arquitectura: Contexto

Identificar sistema que construirán, actores y sistemas externos.Definición textual editable del diagrama, gráfico y una descripción de cada elemento y relación.

Elementos:

Sistema de gestión del taller: es el sistema que vamos a construir, un backend de microservicios que gestiona las órdenes de trabajo desde el ingreso del vehículo hasta su entrega.

Jefe de taller: usuario del sistema. Registra clientes y vehículos, abre órdenes de trabajo, presupuesta, registra la decisión del cliente, gestiona repuestos y registra la entrega.

Mecánico: usuario del sistema. Registra el VIN, los desperfectos, los repuestos y las anotaciones nuevas, y finaliza el trabajo.

Cliente: persona externa que no usa el sistema. Solo recibe por WhatsApp el presupuesto y el aviso de retiro.

Keycloak: proveedor de identidad externo. Autentica a los usuarios, emite los tokens JWT y maneja los roles.

NHTSA vPIC API: API pública que devuelve los datos técnicos del vehículo a partir del VIN.

WhatsApp Cloud API: API de Meta que envía los mensajes de WhatsApp al cliente usando plantillas.

Relaciones:

Jefe de taller y mecánico con Keycloak: inician sesión y obtienen un token con su rol.

Jefe de taller con el sistema: gestiona órdenes, presupuestos, repuestos y entregas por HTTPS con JSON.

Mecánico con el sistema: registra el diagnóstico y el avance de la reparación por HTTPS con JSON.

Sistema con Keycloak: valida el token y el rol en cada operación.

Sistema con vPIC: consulta los datos del vehículo cuando se registra el VIN; si falla, se cargan a mano.

Sistema con WhatsApp Cloud API: envía el presupuesto y el aviso de retiro; si falla, se reintenta.

WhatsApp Cloud API con el cliente: entrega los mensajes en el WhatsApp del cliente.

definicion textual:

```
@startuml C4_Contexto_Taller
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Context.puml

title Diagrama de Contexto - Sistema de gestión del taller mecánico
```

Person(jefe, "Jefe de taller", "Registra clientes y vehículos, abre OT, presupuesta, registra decisiones, gestiona repuestos y entregas")
Person(mecanico, "Mecánico", "Registra VIN, desperfectos, repuestos y anotaciones, y finaliza trabajos")
Person_Ext(cliente, "Cliente", "No usa el sistema. Recibe el presupuesto y el aviso de retiro por WhatsApp")

System(sistema, "Sistema de gestión del taller", "Backend de microservicios que gestiona las órdenes de trabajo del taller")

System_Ext(keycloak, "Keycloak", "Identity Provider: autenticación OIDC, tokens JWT y roles")
System_Ext(vpic, "NHTSA vPIC API", "Decodifica el VIN y devuelve los datos técnicos del vehículo")
System_Ext(whatsapp, "WhatsApp Cloud API", "Envía mensajes de WhatsApp con plantillas")

Rel(jefe, sistema, "Gestiona OT, presupuestos, repuestos y entregas", "HTTPS/JSON")
Rel(mecanico, sistema, "Registra el diagnóstico y el avance de la reparación", "HTTPS/JSON")
Rel_L(jefe, keycloak, "Inicia sesión", "OIDC")
Rel_R(mecanico, keycloak, "Inicia sesión", "OIDC")
Rel(sistema, keycloak, "Valida tokens y roles", "OIDC/JWT")
Rel(sistema, vpic, "Consulta datos del vehículo por VIN", "HTTPS/JSON")
Rel(sistema, whatsapp, "Envía presupuesto y aviso de retiro", "HTTPS/JSON")
Rel(whatsapp, cliente, "Entrega los mensajes", "WhatsApp")

@enduml c4-contexto (1).puml

6- Diagrama C4 - Nivel 2 Contenedores

Sección 6 - Arquitectura: Contenedores

Diagrama de contenedores. Identificación de microservicios, apps, BD y demás unidades ejecutables o almacenes relevantes.

Definición textual:

@startuml C4_Contenedores_Taller

!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml

title Diagrama de Contenedores - Sistema de gestión del taller mecánico

Person(jefe, “Jefe de taller”, “Gestiona clientes, vehículos, OT, presupuestos, repuestos y entregas”)

Person(mecanico, “Mecánico”, “Registra VIN, desperfectos, repuestos y avance de la reparación”)

Person_Ext(cliente, “Cliente”, “No usa el sistema. Recibe avisos por WhatsApp”)

System_Ext(keycloak, “Keycloak”, “Identity Provider: OIDC, JWT y roles”)

System_Ext(vpic, “NHTSA vPIC API”, “Datos técnicos del vehículo por VIN”)

System_Ext(whatsapp, “WhatsApp Cloud API”, “Envío de mensajes con plantillas”)

System_Boundary(taller, “Sistema de gestión del taller (Docker Compose)”) {

Container(gateway, “API Gateway”, “Java, Spring Cloud Gateway”, “Punto de entrada único. Enruta las solicitudes y valida el JWT y el rol por ruta”)

Container(msVehiculos, “ms-clientes-vehiculos”, “Java, Spring Boot”, “Clientes, vehículos por patente y datos técnicos del VIN”)
Container(msOrdenes, “ms-ordenes”, “Java, Spring Boot”, “OT, estados, desperfectos, repuestos, presupuestos, decisiones y entregas”)
Container(msNotif, “ms-notificaciones”, “Java, Spring Boot”, “Plantillas, envío por WhatsApp, historial y reintentos”)
ContainerDb(dbVehiculos, “BD clientes-vehiculos”, “PostgreSQL”, “Esquema de clientes y vehículos”)
ContainerDb(dbOrdenes, “BD ordenes”, “PostgreSQL”, “Esquema de OT, presupuestos y repuestos”)
ContainerDb(dbNotif, “BD notificaciones”, “PostgreSQL”, “Esquema de mensajes enviados y pendientes”)
}
Rel(jefe, keycloak, “Inicia sesión”, “OIDC”)
Rel(mecanico, keycloak, “Inicia sesión”, “OIDC”)
Rel(jefe, gateway, “Usa la API”, “HTTPS/JSON + JWT”)
Rel(mecanico, gateway, “Usa la API”, “HTTPS/JSON + JWT”)
Rel(gateway, keycloak, “Valida el JWT”, “JWKS”)
Rel(gateway, msVehiculos, “Enruta clientes y vehículos”, “REST/JSON”)
Rel(gateway, msOrdenes, “Enruta OT y presupuestos”, “REST/JSON”)
Rel(msOrdenes, msVehiculos, “Consulta vehículo y registra VIN”, “REST/JSON”)
Rel(msOrdenes, msNotif, “Solicita aviso de presupuesto o retiro”, “REST/JSON”)
Rel(msVehiculos, vpic, “Decodifica el VIN”, “HTTPS/JSON”)
Rel(msNotif, whatsapp, “Envía el mensaje”, “HTTPS/JSON”)
Rel(whatsapp, cliente, “Entrega el mensaje”, “WhatsApp”)
Rel(msVehiculos, dbVehiculos, “Lee y escribe”, “JDBC”)
Rel(msOrdenes, dbOrdenes, “Lee y escribe”, “JDBC”)
Rel(msNotif, dbNotif, “Lee y escribe”, “JDBC”)
@enduml

Contenedores:

API Gateway: hecho con Java y Spring Cloud Gateway. Es el único punto de entrada al sistema. Recibe las solicitudes del jefe de taller y del mecánico, valida el token JWT y el rol, y las enruta al microservicio que corresponde.
ms-clientes-vehiculos: microservicio en Java con Spring Boot. Gestiona los clientes y los vehículos por patente, y guarda los datos técnicos obtenidos del VIN. Es el que se integra con la API vPIC.
ms-ordenes: microservicio en Java con Spring Boot. Es el centro del proceso: gestiona las órdenes de trabajo y sus estados, los desperfectos, los repuestos, los presupuestos, las decisiones del cliente y la entrega. Aplica la mayoría de las reglas de negocio.
ms-notificaciones: microservicio en Java con Spring Boot. Arma los mensajes a partir de plantillas, los envía por WhatsApp, guarda el historial y reintenta los que fallan. Está separado para que una falla de WhatsApp no frene las órdenes.
BD clientes-vehiculos, BD ordenes y BD notificaciones: bases de datos PostgreSQL, una por

microservicio. Cada servicio accede solo a la suya.

Elementos externos (los mismos del diagrama de contexto): jefe de taller y mecánico como usuarios; cliente como persona externa que solo recibe avisos; Keycloak como proveedor de identidad; NHTSA vPIC API para los datos del vehículo; y WhatsApp Cloud API para enviar los mensajes.

Relaciones:

Jefe de taller y mecánico con Keycloak: inician sesión con OIDC y obtienen un token JWT con su rol.

Jefe de taller y mecánico con el API Gateway: usan la API por HTTPS con JSON, enviando el token en cada solicitud.

API Gateway con Keycloak: valida la firma del token con las claves públicas de Keycloak (JWKS).

API Gateway con ms-clientes-vehiculos y ms-ordenes: enruta cada solicitud al microservicio correspondiente por REST con JSON.

ms-ordenes con ms-clientes-vehiculos: consulta los datos del vehículo y le pide registrar el VIN cuando el mecánico lo carga.

ms-ordenes con ms-notificaciones: le pide enviar el aviso cuando se genera un presupuesto o cuando el vehículo queda listo para retirar.

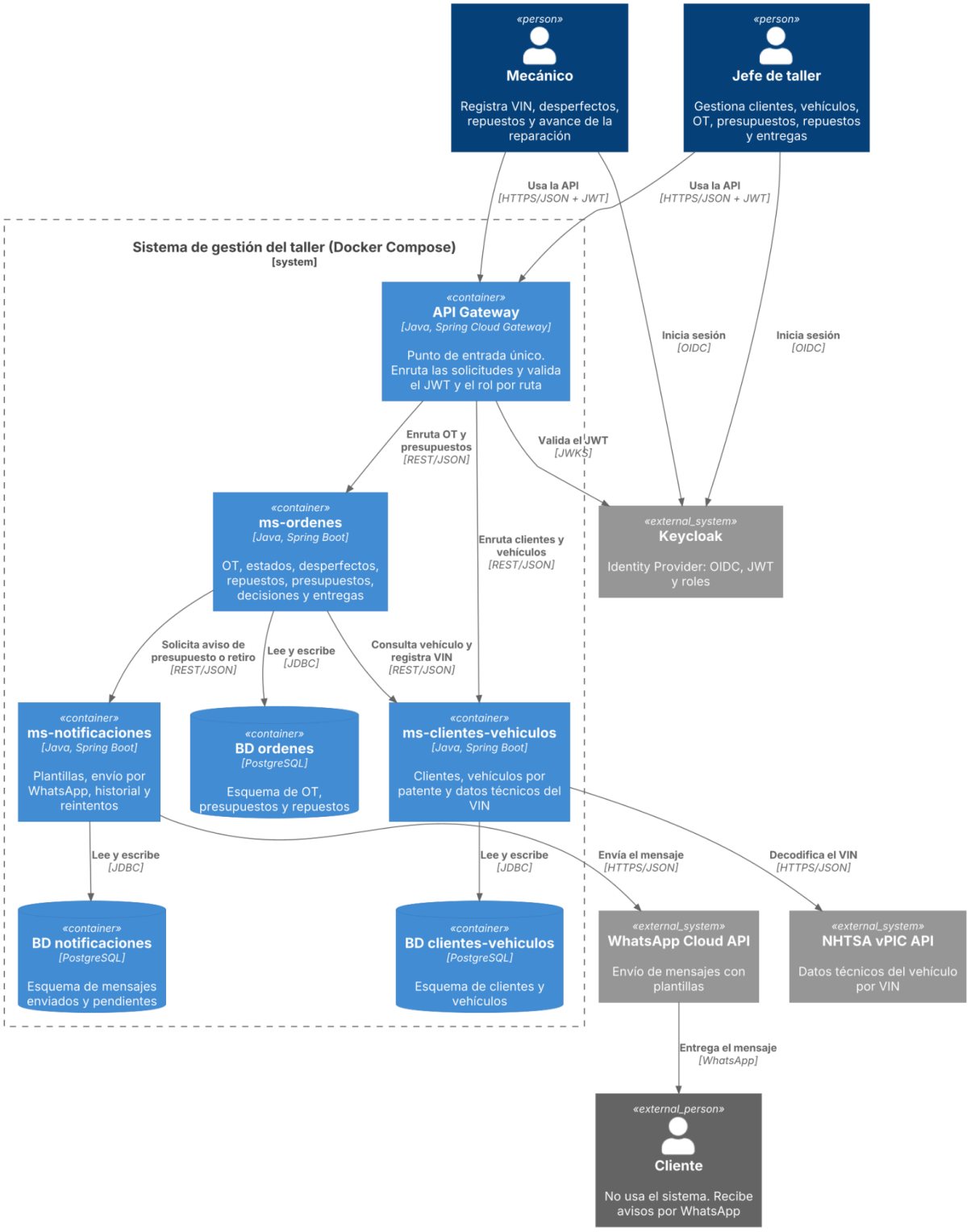
ms-clientes-vehiculos con NHTSA vPIC: decodifica el VIN; si la API no responde, los datos se cargan a mano.

ms-notificaciones con WhatsApp Cloud API: envía el mensaje; si falla, queda pendiente y se reintenta.

WhatsApp Cloud API con el cliente: entrega el mensaje en el WhatsApp del cliente.

Cada microservicio con su base de datos: lee y escribe su propia información por JDBC.

Diagrama de Contenedores - Sistema de gestión del taller mecánico



7- Recursos y endpoints por microservicio

Sección 7 - Arquitectura: Api´s

Tabla por cada ms con todos los endpoints comprometidos.
Columnas: Microservicio | *Recurso* | *Método HTTP* | *Endpoint* | *Operación de negocio* | *Seguridad.*

ms-clientes-vehículos

Microservicio	Recurso	Método HTTP	Endpoint	Operación de negocio	Seguridad
ms-clientes-vehiculos	Clientes	POST	/api/clientes	Registrar un cliente del taller	Jefe de taller
ms-clientes-vehiculos	Clientes	GET	/api/clientes	Listar los clientes registrados	Jefe de taller, Mecánico
ms-clientes-vehiculos	Clientes	GET	/api/clientes/{id}	Consultar los datos de un cliente	Jefe de taller, Mecánico
ms-clientes-vehiculos	Vehículos	POST	/api/vehiculos	Registrar un vehículo de un cliente validando la patente	Jefe de taller
ms-clientes-vehiculos	Vehículos	GET	/api/vehiculos?patente={patente}	Buscar un vehículo por su patente	Jefe de taller, Mecánico
ms-clientes-vehiculos	Vehículos	GET	/api/vehiculos/{id}	Consultar los datos de un vehículo	Jefe de taller, Mecánico
ms-clientes-vehiculos	Vehículos	PUT	/api/vehiculos/{id}	Corregir la marca, el modelo o el año de un vehículo	Jefe de taller
ms-clientes-vehiculos	Vehículos	PUT	/api/vehiculos/{id}/vin	Registrar el VIN y obtener los datos técnicos desde vPIC	Mecánico (uso interno desde ms-ordenes)

ms-ordenes

Microservicio	Recurso	Método HTTP	Endpoint	Operación de negocio	Seguridad
ms-ordenes	Órdenes de trabajo	POST	/api/ordenes	Abrir una orden de trabajo y asignar un mecánico	Jefe de taller
ms-ordenes	Órdenes de trabajo	GET	/api/ordenes?estado={estado}	Listar órdenes por estado	Jefe de taller (todas), Mecánico (solo las asignadas)
ms-ordenes	Órdenes de trabajo	GET	/api/ordenes/{id}	Consultar el detalle e historial de una orden	Jefe de taller, Mecánico asignado
ms-ordenes	Órdenes de trabajo	POST	/api/ordenes/{id}/vin	Identificar el vehículo por VIN e iniciar el diagnóstico	Mecánico asignado
ms-ordenes	Desperfectos	POST	/api/ordenes/{id}/desperfectos	Registrar un desperfecto detectado; si la orden está en reparación, se registra como adicional	Mecánico asignado
ms-ordenes	Repuestos	POST	/api/ordenes/{ordenId}/repuestos	Registrar un repuesto necesario; si la orden está en reparación, se registra como adicional	Mecánico asignado
ms-ordenes	Presupuestos	POST	/api/ordenes/{ordenId}/presupuestos	Generar el presupuesto (inicial o adicional) y avisar al cliente por WhatsApp	Jefe de taller
ms-ordenes	Presupuestos	POST	/api/ordenes/{ordenId}/presupuestos/{presupuestoId}/decision	Registrar si el cliente aceptó o rechazó el presupuesto	Jefe de taller
ms-ordenes	Repuestos	POST	/api/ordenes/{ordenId}/repuestos/pedido	Registrar el pedido de los repuestos aceptados	Jefe de taller
ms-ordenes	Repuestos	POST	/api/ordenes/{ordenId}/repuestos/{repuestoId}/recepcion	Registrar la llegada de un repuesto y habilitar la reparación cuando llegan todos	Jefe de taller
ms-ordenes	Órdenes de trabajo	POST	/api/ordenes/{id}/finalizar	Dar por terminado el trabajo y avisar al cliente que puede retirar	Mecánico asignado

| ms-ordenes | Órdenes de trabajo | POST | /api/ordenes/{id}/entrega | Registrar la entrega del vehículo y cerrar la orden | Jefe de taller |

ms-notificaciones

Microservicio	Recurso	Método HTTP	Endpoint	Operación de negocio	Seguridad
ms-notificaciones	Notificaciones	POST	/api/notificaciones	Enviar al cliente por WhatsApp el aviso de presupuesto o de vehículo listo para retirar	Jefe de taller, Mecánico (uso interno desde ms-ordenes)
ms-notificaciones	Notificaciones	GET	/api/notificaciones?ordenId={ordenId}	Consultar el historial de avisos enviados de una orden	Jefe de taller

8- DER y decisiones de persistencia

Sección 8 - Arquitectura: Persistencia

Diagrama de Entidad-Relación de BD. Con definición textual editable y breve explicación. Explicar que ms administra cada conjunto de datos y principales decisiones de persistencia.

Base clientes_vehiculos (administrada por ms-clientes-vehiculos)

Guarda los clientes del taller y sus vehículos. Un cliente puede tener muchos vehículos, y cada vehículo pertenece a un solo cliente. La patente es única. El vehículo también guarda los datos técnicos obtenidos con el VIN desde la API vPIC, para no consultarla cada vez, y dos marcas de control: si esos datos fueron verificados y si la marca devuelta por vPIC coincide con la registrada.

```
erDiagram
    CLIENTES ||--o{ VEHICULOS : "es dueño de"

    CLIENTES {
        bigint id PK
        varchar(60) nombre
        varchar(60) apellido
        varchar(20) telefono "para WhatsApp"
        varchar(100) email
    }

    VEHICULOS {
        bigint id PK
        varchar(7) patente UK
        varchar(40) marca
        varchar(60) modelo
        int anio
        varchar(17) vin
        varchar(40) marca_vpic "devuelta por vPIC"
        varchar(60) motor
        varchar(40) combustible
        boolean datos_verificados
        boolean marca_inconsistente
        bigint cliente_id FK
    }
```

Base ordenes (administrada por ms-ordenes)

Guarda las órdenes de trabajo y todo lo que se registra sobre ellas:

- Una orden tiene muchos desperfectos, muchos repuestos y muchos presupuestos.
- Cada presupuesto incluye un conjunto de desperfectos y repuestos.
- Un desperfecto o repuesto pertenece a un solo presupuesto, o a ninguno mientras todavía no se presupuestó.

```
erDiagram
    ORDENES_TRABAJO ||--o{ DESPERFECTOS : "registra"
    ORDENES_TRABAJO ||--o{ REPUESTOS_ORDEN : "requiere"
    ORDENES_TRABAJO ||--o{ PRESUPUESTOS : "tiene"
    PRESUPUESTOS |o--o{ DESPERFECTOS : "incluye"
    PRESUPUESTOS |o--o{ REPUESTOS_ORDEN : "incluye"

    ORDENES_TRABAJO {
        bigint id PK
        bigint vehiculo_id "ref. lógica a ms-clientes-vehiculos"
        varchar(50) mecanico_asignado "usuario de Keycloak"
        varchar(30) estado
        timestamp fecha_apertura
        timestamp fecha_entrega
    }

    DESPERFECTOS {
        bigint id PK
        varchar(10) origen "ESCANER o MANUAL"
        varchar(5) codigo_obd
        varchar(500) descripcion
        boolean adicional
        timestamp fecha_registro
        bigint orden_id FK
        bigint presupuesto_id FK "nullable"
    }

    REPUESTOS_ORDEN {
        bigint id PK
        varchar(150) descripcion
        int cantidad
        numeric(12,2) precio_unitario
        varchar(15) estado
        boolean adicional
        bigint orden_id FK
        bigint presupuesto_id FK "nullable"
    }

    PRESUPUESTOS {
        bigint id PK
        varchar(10) tipo "INICIAL o ADICIONAL"
        numeric(12,2) monto_repuestos
        numeric(6,2) horas_mano_obra
        numeric(12,2) tarifa_hora
```

```
    numeric(12,2) monto_mano_obra
    numeric(12,2) otros_cargos
    numeric(12,2) total
    date fecha_emision
    date fecha_vencimiento
    varchar(10) decision
    varchar(20) canal_decision
    varchar(50) registrada_por
    timestamp fecha_decision
    bigint orden_id FK
  }
```

Base notificaciones (administrada por ms-notificaciones)

Guarda el historial de los avisos de WhatsApp enviados al cliente, con su estado y la cantidad de intentos, para poder reintentar los que fallaron. Tiene una sola entidad, que se relaciona con la orden de trabajo por referencia lógica.

```
erDiagram
  NOTIFICACIONES {
    bigint id PK
    bigint orden_id "ref. lógica a ms-ordenes"
    varchar(20) tipo "PRESUPUESTO o VEHICULO_LISTO"
    varchar(20) telefono
    varchar(120) nombre_cliente
    numeric(12,2) monto
    varchar(10) estado "PENDIENTE, ENVIADA o FALLIDA"
    int intentos
    varchar(500) ultimo_error
    timestamp fecha_creacion
    timestamp fecha_envio
  }
```

Decisiones de Persistencia:
Autonomía de bases de datos (Database per Service): Cada microservicio posee su propio esquema/base de datos en PostgreSQLy gestiona sus entidades mediante Spring Data JPA / Hibernate y JDBC. Ningún servicio realiza consultas SQL ni JOIN sobre tablas de otro microservicio.

Referencias lógicas en lugar de claves foráneas entre servicios: La tabla ordenes_trabajo almacena vehiculo_id (y patente) como referencia lógica al vehículo registrado en ms-clientes-vehiculos, y la tabla notificaciones almacena orden_trabajo_id como referencia lógica a la OT de ms-ordenes. La validación de existencia y sincronización de datos entre servicios se resuelve en la capa de aplicación mediante llamadas REST síncronas (VehiculosClient y NotificacionesClient).

Trazabilidad de actores y reglas de negocio en las tablas:
-En ordenes_trabajo se persiste el identificador del mecanicoAsignado extraído del token JWT de Keycloak para validar en cada operación de diagnóstico o reparación que solo dicho mecánico modifique la orden (RN-03).
-Una misma OrdenTrabajo puede tener múltiples registros en presupuestos (1:N), lo que permite conservar los presupuestos vencidos cuando se emite uno nuevo (RN-06) y separar el presupuesto inicial de los presupuestos adicionales generados durante la reparación (RN-08).

9- Librería común y reutilización

Sección 9 - Arquitectura: Librería común

Implementación de una librería común. Nombre, propósito, componentes que contendrá, servicios que la utilizarán y mecanismo de distribución o dependencia. Justificar qué elementos son realmente reutilizables y cuáles deben permanecer dentro de cada microservicio.

Decidimos implementar una librería común llamada taller-commons (groupId utnfc.isi.back, versión 1.0.0). Su propósito es reunir el código transversal que varios servicios necesitan usar de la misma manera, para no copiarlo entre proyectos.
Contiene tres partes. Primero, el manejo de errores: una excepción de negocio con el código de la regla incumplida, un formato estándar de error y un manejador global base que cada servicio extiende. Segundo, la integración con Keycloak: la conversión de los roles del token a roles de Spring Security y una utilidad para obtener el usuario y el token actual. Tercero, los validadores de patente y VIN. La usan ms-clientes-vehiculos, ms-ordenes, ms-notificaciones y el API Gateway. Se empaqueta como JAR, se instala en el repositorio Maven local con mvn install y cada servicio la declara como dependencia en su pom.xml.
Incluimos solo lo que realmente se reutiliza. El formato de error lo producen todos los servicios y ms-ordenes lo lee al recibir errores de ms-clientes-vehiculos. La conversión de roles tiene que ser idéntica en el gateway y en los servicios para que un mismo token dé los mismos permisos. Y el VIN lo validan dos servicios. Dentro de cada microservicio quedan las entidades, los repositorios, los DTOs, las reglas de negocio y los clientes de las APIs externas, porque dependen del dominio de cada uno y compartirlos acoplaría los servicios.

10- Seguridad y Autorización

Sección 10 - Implementación y verificación

Mecanismos de autenticación previstos, roles o permisos, recursos protegidos y comportamientos ante accesos no autorizados. Explicar cómo comprobar al menos un acceso permitido y uno rechazado.

La autenticación la maneja Keycloak con OpenID Connect. El usuario inicia sesión y recibe un token JWT con su usuario y sus roles, que envía en cada solicitud. El gateway y los microservicios verifican la firma y la vigencia del token con las claves públicas de Keycloak.

Hay dos roles. JEFE_TALLER gestiona clientes, vehículos, órdenes, presupuestos, repuestos y entregas. MECANICO registra el VIN, los desperfectos y los repuestos, y finaliza el trabajo, pero solo en las órdenes que tiene asignadas.

Todos los recursos bajo /api están protegidos en dos niveles. El API Gateway valida el token y aplica permisos por ruta, y cada microservicio vuelve a validar el token y el rol de cada operación. Si no hay token, o es inválido o está vencido, el sistema responde 401. Si el rol no alcanza, o el mecánico intenta operar sobre una orden que no es suya, responde 403.

Para comprobarlo, se pide un token a Keycloak con el usuario jefe y otro con mecanico1. Con el token del jefe, POST /api/clientes registra el cliente y responde 201. Con el token de mecanico1, la misma solicitud responde 403, y sin token responde 401.

11- Ejecución y Despliegue

Sección 10 - Implementación y verificación

Indicar tecnologías y versiones principales, dependencias de infraestructura, configuración por entorno y procedimiento de arranque de solución. Describir qué archivos o instrucciones permitirán ejecutar el backend completo, incluyendo contenedores si corresponde.

Usamos Java 17, Spring Boot 3.2.5 y Maven, con Spring Cloud Gateway para el API Gateway, Spring Security, Spring Data JPA y springdoc-openapi 2.3.0. La infraestructura es PostgreSQL 16 y Keycloak 24, ambos en contenedores. Las APIs de vPIC y WhatsApp son externas al despliegue.

La configuración se hace con variables de entorno. Cada servicio trae valores por defecto para correr en local, y Docker Compose los reemplaza para el entorno de contenedores. Las contraseñas y el token de WhatsApp van en un archivo .env, que se arma a partir de .env.example.

Para levantar todo hacen falta JDK 17, Maven y Docker. Primero se corre build.cmd o build.sh, que instala la librería común y empaqueta los servicios. Después se ejecuta docker compose up --build. Esto lo hacen posible el docker-compose.yml, un Dockerfile por servicio, el init.sql que crea las bases y el realm de Keycloak con los roles y usuarios de prueba. La API queda disponible en el gateway, en el puerto 8080.

12- Alcance acordado, desafíos y exclusiones

Sección 11 - Compromiso del equipo

Sintetizar el compromiso completo del equipo: Proceso central que quedará funcionando, funcionalidades obligatorias, integraciones relevantes, desafíos técnicos o funcionales y exclusiones expresas. Identificar riesgos o dependencias que puedan afectar el alcance. El compromiso declarado será la base de la validación de la cátedra y criterios específicos de corrección.

Vamos a entregar una colección de Postman en el repositorio, junto con su archivo de entorno. Las solicitudes están organizadas en carpetas que siguen el proceso: autenticación, clientes y vehículos, diagnóstico, presupuesto, repuestos y reparación, entrega, y casos negativos.

El entorno tiene la URL del gateway y la de Keycloak. Las primeras solicitudes piden los tokens del jefe y de los mecánicos, y cada respuesta guarda los ids que usan las siguientes. Los únicos datos iniciales son los usuarios de Keycloak: todo lo demás lo crea la propia colección, que se ejecuta en orden con el Collection Runner.

El escenario positivo recorre el proceso completo, desde el alta del vehículo hasta la entrega. Los escenarios negativos prueban las reglas de negocio, como patente o VIN inválidos, un mecánico no asignado o un presupuesto sin aceptar, y los accesos sin token o con un rol incorrecto. Para las integraciones, se prueba un VIN que vPIC no reconoce, con carga manual, y un envío de WhatsApp que falla y queda pendiente para reintentar.

13- Criterios de verificación propuestos

Sección 11 - Compromiso del equipo

Describir de 3 a 8 comprobaciones observables que permitan verificar el cumplimiento del compromiso: Condición inicial, acción a ejecutar y resultado esperado. Relacionar cada comprobación con las funcionalidades, reglas o endpoints que correspondan.

Nos comprometemos a que funcione el ciclo completo de una orden de trabajo, desde el ingreso del vehículo hasta su entrega. Eso incluye el registro de clientes y vehículos, el diagnóstico, el presupuesto con la decisión del cliente, la gestión de repuestos y la reparación, aplicando las reglas de negocio definidas.

Las integraciones son vPIC, para obtener los datos del vehículo por VIN, y WhatsApp, para avisarle al cliente. El principal desafío es coordinar tres microservicios y que el proceso siga funcionando aunque una API externa falle.

Quedan afuera las acciones del cliente en el sistema, los turnos, la facturación, el stock, las sucursales y el frontend.

Hay dos riesgos: que vPIC no reconozca algunos autos argentinos, y por eso está prevista la carga manual, y que WhatsApp necesite plantillas aprobadas por Meta, y por eso los envíos se pueden simular.

.----------

1. Registro del VIN. Con una orden en estado Recibida, el mecánico asignado registra un VIN válido. Se espera que la orden pase a En diagnóstico y que el vehículo quede con los datos de vPIC. Se relaciona con F03, RN-04 y POST /api/ordenes/{id}/vin.
2. Generación del presupuesto. Con una orden en diagnóstico que tiene desperfectos y repuestos cargados, el jefe genera el presupuesto. Se espera que se calcule el total con vencimiento a 7 días, que la orden pase a Presupuestada y que se avise al cliente por WhatsApp. Se relaciona con F06, RN-06 y POST /api/ordenes/{id}/presupuestos.
3. Sin aceptación no se avanza. Con un presupuesto que todavía no tiene decisión, el jefe intenta pedir los repuestos. Se espera que el sistema lo rechace con el código RN-07. Se relaciona con F08, RN-07 y POST /api/ordenes/{id}/repuestos/pedido.
4. Solo opera el mecánico asignado. Con una orden en reparación asignada a mecanico1, mecanico2 intenta finalizarla. Se espera que el sistema responda 403 con el código RN-03. Se relaciona con F10, RN-03 y POST /api/ordenes/{id}/finalizar.